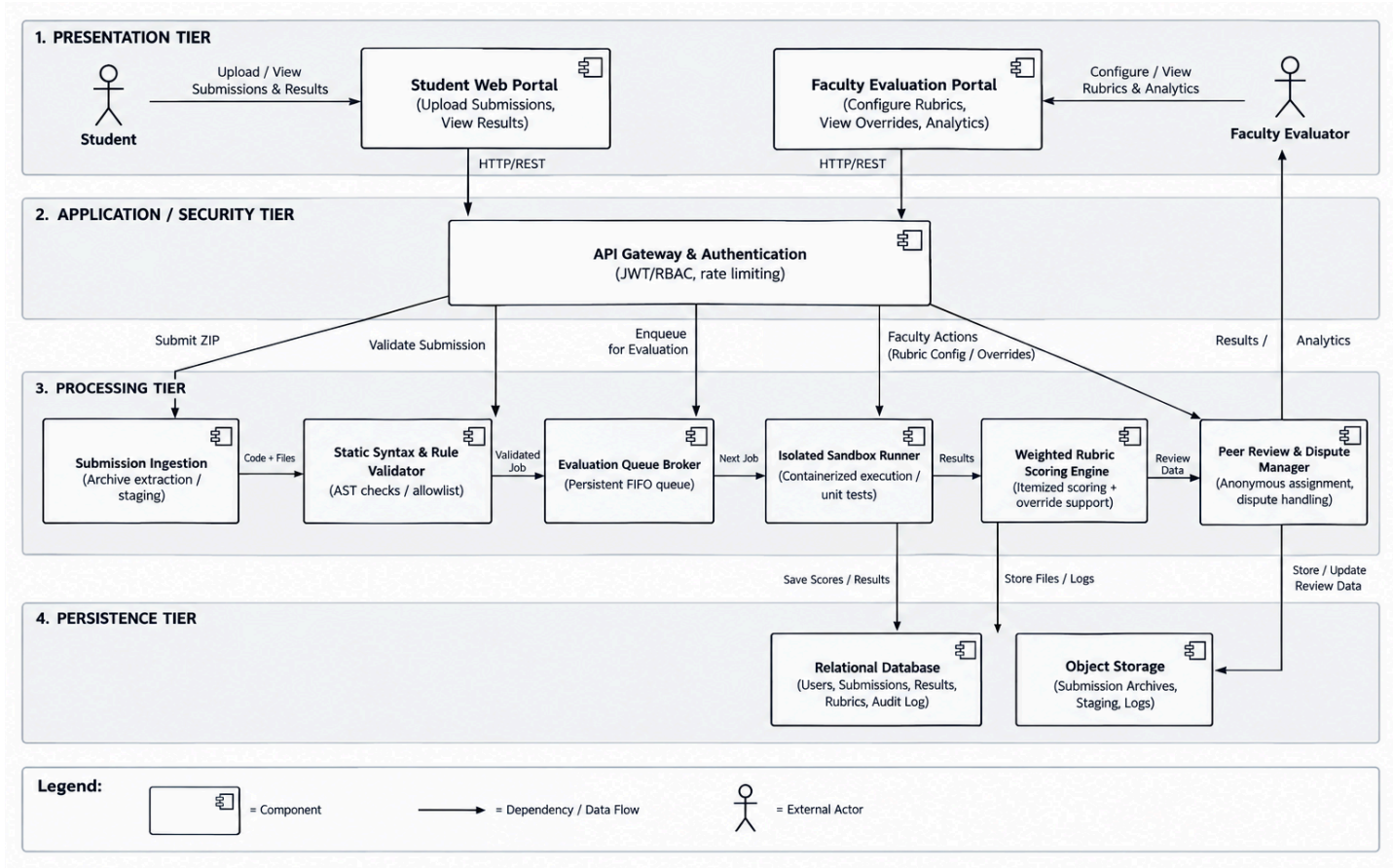


1. Component Diagram



2. Architectural Justification

2.1 Architectural Pattern Selection

The Automated Rubric Assignment Evaluator uses an Asynchronous Event-Driven Architecture (Worker Queue Pattern) paired with a Microkernel (Plugin-based) Execution Engine and an Edge API Gateway. This pattern balances high-volume HTTP submission ingestion with compute-intensive, untrusted code execution.

- **Asynchronous Queue-Based Ingestion:** The web tier validates uploads, stores archives in object storage, enqueues evaluation events, and immediately returns an HTTP 202 status. This decouples client connections from long-running tasks, preventing web thread pool exhaustion during submission spikes.
- **Microkernel Execution Engine:** The core platform handles workflows, scoring, and peer review distribution, while language-specific runtimes (PyTest, JUnit, GoogleTest) run inside pluggable, containerized sandboxes.

Edge Gateway & Layered Security: The API Gateway acts as a single point of ingress, enforcing JWT authentication, Role-Based Access Control (RBAC), and rate limiting to protect against DoS attacks.

2.2 Architectural Trade-off Analysis

To justify this architectural choice, competing architectural styles were evaluated against the functional and non-functional requirements of Problem Statement #02:

Architectural Pattern	Advantages	Disadvantages & Liabilities for PS #02
Traditional Monolithic 3-Tier (MVC)	Simple deployment, single codebase, low network overhead, direct transactional database access.	Rejected: Synchronous request-response model binds web threads to execution runs. Under 100 concurrent submissions (NFR-001), thread pools starve, HTTP connections timeout, and memory spikes far exceed 1.5 GB. Malicious student code can compromise the shared host process.
Distributed Microservices Pattern	Independent service scalability, technological flexibility, isolated fault domains.	Rejected: Excessive infrastructure overhead (service mesh, distributed tracing, network serialization latency). Cumulative memory baseline for multiple microservice JVM/Node runtimes exceeds ~3.0 GB, directly violating NFR-001 (<1.5 GB).
Event-Driven Worker Queue + Microkernel Sandbox (Selected)	Absorbs submission bursts gracefully; guarantees zero queue loss; enforces strict memory quotas per worker; completely isolates untrusted execution.	Accepted: Optimal fit. Requires asynchronous client status polling/WebSockets and message broker management, which are easily handled by standard queue tooling.

2.3 Non-Functional Requirements Justification

A. Justification for NFR-001 (Performance & Scalability):

Target: Process up to 100 concurrent submissions without dropping requests or exceeding 1.5 GB memory footprint.

- **Decoupled Ingestion:** Asynchronous I/O threads quickly validate uploads, stream archives to storage, and enqueue lightweight jobs (~1.2 KB) in under 150ms without blocking web workers.
- **Durable Buffering:** A persistent disk-backed FIFO message broker safely queues peak traffic without dropping requests.

Bounded Memory: System memory is strictly capped across all subsystems to stay within the 1.5 GB limit.

Subsystem / Component	Memory Allocation	Resource Control Mechanism
API Gateway & Ingestion Service	200 MB	Lightweight asynchronous runtime with streaming multipart parsing.

Static Syntax & Rule Validator	150 MB	In-memory AST parsing with transient worker threads and rapid GC.
Evaluation Queue Broker (Redis/RabbitMQ)	150 MB	In-memory queue buffer capped at 10,000 descriptors with disk paging.
Active Sandbox Worker Pool (4 runners × 150 MB)	600 MB	Strict Linux cgroups memory.limit_in_bytes=150M per container.
Scoring Engine & Database Connection Pool	150 MB	HikariCP connection pool capped at 20 active connections.
Host OS Kernel & Network Buffers	250 MB	Host Linux kernel space, socket buffers, shared C libraries.
Total System Peak Footprint	1500 MB (1.5 GB)	Strictly complies with the 1.5 GB non-functional ceiling.

B. Justification for NFR-002 (Security & Execution Isolation):

Target: Safely run untrusted student code inside isolated sandboxes with disabled network access and a 15-second execution timeout.

- **Container Isolation:** Code runs inside gVisor/containerd sandboxes that intercept system calls in user space, preventing kernel exploits or host escapes.
- **Disabled Network Access:** Containers run with isolated network namespaces (loopback only), preventing outbound connections or malicious network traffic.
- **15-Second Watchdog:** A supervisor process terminates containers exceeding 15 seconds via SIGKILL, preventing infinite loops or resource exhaustion.

Read-Only Mounting: Source files are mounted as read-only volumes with temporary file creation restricted to an in-memory 32 MB tmpfs.

2.4 Functional Workflow Justification

- **Static Syntax & Rule Gatekeeping (FR-002):** Invalid or malicious code archives are rejected early to save sandbox resources.
- **Rubric Normalization & Overrides (FR-003):** Ensures scoring weights total 100% and maintains an append-only log for manual grade adjustments.
- **Double-Blind Peer Review (FR-004):** Anonymizes submissions and uses round-robin assignment (k=2) for unbiased evaluations.

Time-Gated Disputes (FR-005): Allows students to submit regrade requests with required rationales during active dispute windows.